

Node.JS_Quick_Revision_Short_Note

<https://gemini.google.com/app/15254f58cca8a044>

User prompt: Solve All The Question and give me All The Question with an Answer Do Not Skip Any Question Solve In All This Question and Explain in Detailed in simple with an Example in Exam and Technical Interview Point of view in Real Wolrd Programming Example # Node.js Technical Interview Questions (1–100) #### Basic → Intermediate → Advanced | Real-World Focused --- ## SECTION 1: BASIC LEVEL (Q1–Q35) *Core concepts every Node.js developer must know* #### Fundamentals 1. What is Node.js? How is it different from a browser's JavaScript runtime? 2. What is the V8 engine, and what role does it play in Node.js? 3. Is Node.js single-threaded or multi-threaded? Explain what this actually means in practice. 4. What is the Event Loop? Why is it central to Node.js's architecture? 5. What is non-blocking I/O? Give a real-world example of why it matters (e.g., handling thousands of concurrent requests). 6. What is the difference between synchronous and asynchronous code in Node.js? 7. What is a callback function? What is "callback hell" and why is it a problem? 8. What is `npm`? What is the difference between `dependencies` and `devDependencies` in `package.json`? 9. What is the difference between `npm install` and `npm ci`? 10. What is the purpose of the `package-lock.json` file? #### Modules 11. What is a module in Node.js? How do you create and export one? 12. What is the difference between CommonJS (`require`/`module.exports`) and ES Modules (`import`/`export`)? 13. What is the difference between `module.exports` and `exports`? 14. What are core (built-in) modules in Node.js? Name a few (`fs`, `http`, `path`, `os`). 15. What is the `require` cache? What happens if you `require` the same module twice? #### Core Modules & Basics 16. How do you create a basic HTTP server in Node.js without Express? 17. What is the `fs` module used for? Give a real-world use case (e.g., reading a config file). 18. What is the difference between `fs.readFile` and `fs.readFileSync`? When would you use each? 19. What is the `path` module used for? Why shouldn't you manually concatenate file paths with strings? 20. What is the `process` object? Give a real-world example of using `process.env`. #### Async Programming Basics 21. What is a Promise? How does it improve on plain callbacks? 22. What are the three states of a Promise? 23. What is `async/await`? How does it relate to Promises under the hood? 24. How do you handle errors in `async/await` code? Why is `try/catch` important here? 25. What is `Promise.all()`? Give a real-world example (e.g., fetching data from multiple APIs simultaneously). #### Express.js Basics 26. What is Express.js and why is it commonly used with Node.js? 27. What is middleware in Express? Give a real-world example (e.g., logging, authentication). 28. What is the difference between `app.use()` and `app.get()`? 29. How do you handle route parameters and query strings in Express? 30. What is the purpose of the `next()` function in Express middleware? #### Basic Concepts (Continued) 31. What is REST API? How would you design basic CRUD routes for a `User` resource? 32. What is JSON and why is it the standard data format for APIs in Node.js? 33. What is the difference between `PUT` and `PATCH` HTTP methods in a real-world update scenario? 34. What are environment variables and why should sensitive data (API keys, DB passwords) never be hardcoded? 35. What is the purpose of the `.env` file and the `dotenv` package? --- ## SECTION 2: INTERMEDIATE LEVEL (Q36–Q70) *Architecture, real-world backend design, common interview traps* #### Event Loop Deep Dive 36. Explain the phases of the Node.js Event Loop (timers, pending callbacks, poll, check, close callbacks). 37. What is the difference between `process.nextTick()` and `setImmediate()`? 38. What is the difference between `setTimeout(fn, 0)` and `setImmediate(fn)`? 39. What is the microtask queue? How do Promises interact with the Event Loop's phases? 40. Why can a long-running synchronous function block the entire Node.js server? Give a real-world example (e.g., a heavy JSON parse or CPU-bound loop). #### Streams & Buffers 41. What are Streams in Node.js? Why are they useful for handling large files (e.g., video upload)? 42. What are the four types of streams (Readable, Writable, Duplex, Transform)? 43. What is backpressure in streams, and why does it matter for performance? 44. What is a Buffer in Node.js? Why is it needed when working with binary data? 45. How would you

efficiently stream a large CSV file to a client instead of loading it fully into memory? ### Error Handling & Debugging 46. What is the difference between operational errors and programmer errors in Node.js? 47. How do you handle uncaught exceptions globally (`process.on('uncaughtException')`)? Is this a good practice in production? 48. How do you handle unhandled Promise rejections (`process.on('unhandledRejection')`)? 49. How would you design a centralized error-handling middleware in an Express application? 50. What is the difference between throwing an error and passing it via `next(err)` in Express? ### Real-World Backend Design 51. How would you design authentication in a Node.js app using JWT? What are the trade-offs vs session-based auth? 52. How would you implement rate limiting on an API to prevent abuse (e.g., login brute force)? 53. How do you handle file uploads in Express (e.g., using `multer`)? What real-world validations are needed? 54. How would you structure a scalable Node.js project (routes, controllers, services, models)? Why does this separation matter? 55. What is middleware chaining, and how would you use it for request validation, authentication, and logging in sequence? 56. How do you connect Node.js to a database (SQL vs NoSQL)? What are connection pooling and why is it important? 57. How would you prevent SQL Injection / NoSQL Injection in a Node.js application? 58. How would you implement pagination for an API returning thousands of records? 59. How would you design a caching layer (e.g., Redis) to reduce database load for frequently accessed data? 60. How do you implement CORS in an Express app, and why is it needed for frontend-backend communication? ### Testing & Quality 61. What is unit testing vs integration testing in a Node.js context? 62. What tools are commonly used for testing Node.js applications (Jest, Mocha, Chai, Supertest)? 63. How would you mock a database call in a unit test? 64. What is the purpose of a linter (ESLint) in a real-world Node.js team project? 65. How do you handle environment-specific configurations (dev, staging, production)? ### Security 66. What are common security vulnerabilities in Node.js apps (e.g., XSS, CSRF, injection attacks)? 67. Why should you use `helmet` middleware in an Express app? 68. How do you securely store and compare passwords (e.g., `bcrypt`)? 69. What is the principle of least privilege, and how does it apply to database user permissions in a Node.js app? 70. How would you protect an API from a DDoS-style abuse pattern at the application level? --- ## SECTION 3: ADVANCED LEVEL (Q71–Q100) *Scalability, architecture, internals, system design thinking* ### Concurrency & Performance 71. Node.js is single-threaded, so how does it handle multiple concurrent requests efficiently? Explain the role of `libuv`. 72. What is the `Worker Threads` module, and when would you use it over the main Event Loop (e.g., CPU-intensive image processing)? 73. What is the `Cluster` module, and how does it help utilize multi-core CPUs? 74. What is the difference between horizontal scaling and vertical scaling for a Node.js application? 75. How would you profile and identify a memory leak in a Node.js production application? 76. What is a common cause of memory leaks in Node.js (e.g., unbounded caches, global variable accumulation, unclosed listeners)? 77. How does the V8 garbage collector work, and how can poor coding patterns increase GC pressure? 78. How would you handle a CPU-intensive task (e.g., PDF generation, video transcoding) without blocking the Event Loop? ### Architecture & System Design 79. How would you design a real-time chat application using Node.js (WebSockets vs polling)? 80. What is the difference between REST and GraphQL? When would you choose GraphQL for a Node.js backend? 81. How would you design a microservices architecture in Node.js? How do services communicate (REST, message queues, gRPC)? 82. What role do message queues (RabbitMQ, Kafka) play in a Node.js microservices system? Give a real-world use case (e.g., order processing pipeline). 83. How would you implement an event-driven architecture using Node's `EventEmitter`, and how does this differ from a message queue at scale? 84. How would you design a job queue system for background tasks (e.g., sending emails, generating reports) using something like BullMQ? 85. How would you handle distributed transactions across multiple microservices (e.g., Saga pattern)? 86. How would you design an API Gateway in front of multiple Node.js microservices? 87. What is idempotency, and why is it critical when designing payment or order APIs? 88. How would you version an API in a real-world Node.js application to avoid breaking existing clients? ### Advanced Internals 89. What is the difference between `require()` and dynamic `import()`? What are the performance

implications? 90. Explain how Node.js resolves module paths (`node\_modules` lookup algorithm). 91. What is the Event Emitter pattern, and how is it used internally in core modules like `http` and `stream`? 92. What is the difference between deep copy and shallow copy in JavaScript objects, and how does this cause real bugs in Node.js apps (e.g., mutating shared config objects)? 93. How does Node.js handle child processes (`spawn`, `exec`, `fork`)? Give a real-world use case for each. 94. What is the difference between `spawn` and `exec` in terms of buffering output — why does this matter for large output data? 95. What is graceful shutdown, and how would you implement it to properly close DB connections and finish in-flight requests before a server restarts (e.g., during deployment)? 96. How would you implement health checks and readiness/liveness probes for a Node.js service running in Kubernetes? 97. What are the trade-offs of using Docker to containerize a Node.js application in production? 98. How would you implement structured logging and distributed tracing (e.g., correlation IDs) across multiple Node.js microservices? 99. What is Server-Sent Events (SSE), and how does it compare to WebSockets for a real-world use case like live notifications? 100. If you were designing a real-world food delivery backend (like Zomato/Uber Eats) in Node.js — covering order placement, live tracking, payments, and notifications — how would you architect the services, handle real-time updates, and ensure reliability under high load?

Response: Node.js Technical Interview Questions & Answers (1–100)

SECTION 1: BASIC LEVEL (Q1–Q35)

Q1: What is Node.js? How is it different from a browser's JavaScript runtime?

- **Answer:** Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's V8 engine that allows developers to run JavaScript on the server. While both use V8, browser JavaScript interacts with the DOM and lacks file system access, whereas Node.js runs on an operating system, provides full disk/network access, and lacks a DOM.
- **Real-World Example:** Building a backend web server using Node.js to read configuration files and handle database calls, whereas browser JS handles client-side UI button clicks.

Q2: What is the V8 engine, and what role does it play in Node.js?

- **Answer:** V8 is Google's open-source high-performance JavaScript and WebAssembly engine written in C++. Its role in Node.js is to compile human-readable JavaScript code directly into native machine code using Just-In-Time (JIT) compilation for fast execution.
- **Real-World Example:** Converting an arithmetic calculation loop in a Node.js script into optimized CPU machine instructions instantly.

Q3: Is Node.js single-threaded or multi-threaded? Explain what this actually means in practice.

- **Answer:** Node.js user-written JavaScript code executes on a single main thread (the event loop's call stack). However, under the hood, the runtime uses a multi-threaded pool (libuv) and operating system kernels to offload background I/O tasks.
- **Real-World Example:** A server handling 10,000 simultaneous file reads does not freeze because libuv's background thread pool handles the disk operations concurrently while the main thread accepts new connections.

Q4: What is the Event Loop? Why is it central to Node.js's architecture?

- **Answer:** The Event Loop is a continuous orchestration mechanism provided by libuv that monitors the call stack and task queues, enabling non-blocking I/O operations despite JavaScript being single-threaded.
- **Real-World Example:** Handling an incoming HTTP request, delegating the database fetch, and executing the response callback once the data returns without freezing other users.

Q5: What is non-blocking I/O? Give a real-world example of why it matters.

- **Answer:** Non-blocking I/O means execution does not halt while waiting for slow input/output operations (like disk reads or network calls) to finish; instead, the process delegates the task and moves on.
- **Real-World Example:** An e-commerce checkout API processes 500 simultaneous user orders without waiting for individual payment gateway network responses to block subsequent entries.

Q6: What is the difference between synchronous and asynchronous code in Node.js?

- **Answer:** Synchronous code blocks further execution until the current line finishes entirely, whereas asynchronous code initiates a task, registers a callback or promise, and continues executing subsequent lines immediately.
- **Real-World Example:** `fs.readFileSync` locks the thread until a file is completely loaded into memory, while `fs.readFile` reads the file in the background while executing code below it.

Q7: What is a callback function? What is "callback hell" and why is it a problem?

- **Answer:** A callback is a function passed as an argument to another function to be executed once an asynchronous task completes. "Callback hell" (Pyramid of Doom) occurs when multiple dependent asynchronous operations nest callbacks deeply inside one another, making code unmaintainable and hard to debug.
- **Real-World Example:** Nesting user authentication, database lookup, and file writing callbacks inside each other until code indentation shifts heavily to the right.

Q8: What is npm? What is the difference between dependencies and devDependencies in package.json?

- **Answer:** npm (Node Package Manager) is the default package manager for Node.js. `dependencies` are required for the application to run successfully in production, whereas `devDependencies` are packages needed only during development (e.g., testing frameworks, linters).
- **Real-World Example:** Express.js goes into `dependencies` to serve live traffic, while Jest goes into `devDependencies` for running local tests.

Q9: What is the difference between `npm install` and `npm ci`?

- **Answer:** `npm install` installs packages and can modify `package-lock.json` if discrepancies arise, whereas `npm ci` (Clean Install) is designed for automated CI/CD

environments—it deletes `node_modules`, performs a strict clean install directly from `package-lock.json`, and fails if versions drift.

- **Real-World Example:** Running `npm ci` inside a GitHub Actions CI pipeline to ensure reproducible, identical builds every deployment.

Q10: What is the purpose of the `package-lock.json` file?

- **Answer:** It locks down the exact dependency tree and version numbers installed in a project to ensure deterministic builds across different machines and environments.
- **Real-World Example:** Preventing a team member from accidentally installing a breaking patch version of a package that differs from production.

Q11: What is a module in Node.js? How do you create and export one?

- **Answer:** A module is an encapsulated block of code that can share functions or variables using `export` syntax.
- **Real-World Example:** Creating a file `math.js`, defining `const add = (a, b) => a + b;`, and exporting it via `module.exports = { add };`.

Q12: What is the difference between CommonJS (`require/module.exports`) and ES Modules (`import/export`)?

- **Answer:** CommonJS is synchronous and native to older Node.js versions using `require()`, while ES Modules use asynchronous static imports (`import/export`) built into the official JavaScript specification.
- **Real-World Example:** Writing modern frontend-aligned syntax (`import express from 'express'`) instead of legacy CommonJS (`const express = require('express')`).

Q13: What is the difference between `module.exports` and `exports`?

- **Answer:** `exports` is simply a reference point pointing directly to `module.exports`. If you reassign `exports = {}`, you break the link, whereas modifying properties on `exports.name = value` mutates the actual exported object.
- **Real-World Example:** Assigning a raw function directly using `module.exports = myFunction` rather than `exports.myFunction = myFunction`.

Q14: What are core (built-in) modules in Node.js? Name a few (`fs`, `http`, `path`, `os`).

- **Answer:** Core modules are built directly into the Node.js binary without needing external installation via npm.
- **Real-World Example:** Using `const path = (__dirname, 'public')` to dynamically resolve file paths safely across Windows and Linux.

Q15: What is the `require` cache? What happens if you `require` the same module twice?

- **Answer:** Node.js caches modules after their initial loading. When `require()` is called a second time, it pulls from memory instead of reloading and executing the file from disk.

- **Real-World Example:** Requiring a database connection configuration singleton multiple times across different files safely without re-establishing new instances.

Q16: How do you create a basic HTTP server in Node.js without Express?

- **Answer:** By using the native `http` core module's `createServer` method.
- **Real-World Example:**

```
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello World');
}).listen(3000);
```

Q17: What is the `fs` module used for? Give a real-world use case.

- **Answer:** The `fs` (File System) module interacts with the local disk to read, write, update, and delete files.
- **Real-World Example:** Reading an application configuration file (`config.json`) upon server startup.

Q18: What is the difference between `fs.readFile` and `fs.readFileSync`?

- **Answer:** `fs.readFile` is asynchronous and non-blocking, whereas `fs.readFileSync` is synchronous and blocks the main thread until the entire file loads.
- **Real-World Example:** Using `fs.readFileSync` only once during initial script boot-up, but always using `fs.readFile` when handling live user web requests.

Q19: What is the `path` module used for? Why shouldn't you manually concatenate file paths with strings?

- **Answer:** The `path` module handles and normalizes file paths. Manual string concatenation (`/` vs `\`) breaks cross-platform compatibility between Windows and Unix systems.
- **Real-World Example:** Using `path.join(__dirname, 'views', 'index.html')` to ensure compatibility regardless of operating system slash conventions.

Q20: What is the `process` object? Give a real-world example of using `process.env`.

- **Answer:** The `process` object is a global object providing information about, and control over, the current Node.js process.
- **Real-World Example:** Accessing database connection strings securely via `const dbUrl = process.env.DATABASE_URL;`

Q21: What is a Promise? How does it improve on plain callbacks?

- **Answer:** A Promise is an object representing the eventual completion or failure of an asynchronous operation, flattening code structures and preventing callback hell via `.then()` chaining.

- **Real-World Example:** Chaining sequential database queries cleanly without nesting functions.

Q22: What are the three states of a Promise?

- **Answer:** Pending (initial state), Fulfilled (operation completed successfully), and Rejected (operation failed).
- **Real-World Example:** An HTTP fetch request starts as *pending*, becomes *fulfilled* on status 200, or *rejected* if network connectivity drops.

Q23: What is `async/await`? How does it relate to Promises under the hood?

- **Answer:** `async/await` is syntactic sugar built directly on top of Promises that allows writing asynchronous code in a synchronous, readable fashion.
- **Real-World Example:** Using `const data = await fetchData()` instead of `.then(data => ...)`.

Q24: How do you handle errors in `async/await` code? Why is `try/catch` important here?

- **Answer:** Errors are handled by wrapping `await` statements inside `try/catch` blocks. Without it, rejected promises throw uncaught exceptions that can crash applications.
- **Real-World Example:** Catching a database connection timeout error safely to return a 500 JSON response to the user.

Q25: What is `Promise.all()`? Give a real-world example.

- **Answer:** `Promise.all()` accepts an array of promises and runs them concurrently, resolving only when all promises succeed or rejecting immediately if any fail.
- **Real-World Example:** Fetching user profile data, active cart items, and notification logs simultaneously from three different microservices to render a dashboard page quickly.

Q26: What is Express.js and why is it commonly used with Node.js?

- **Answer:** Express.js is a minimal, flexible Node.js web application framework that provides robust routing, middleware integration, and simplified HTTP request handling.
- **Real-World Example:** Building RESTful API endpoints (`app.get('/users', ...)`) without writing verbose native `http` module matching conditions.

Q27: What is middleware in Express? Give a real-world example.

- **Answer:** Middleware functions have access to the request (`req`), response (`res`), and the next middleware function in the application's request-response cycle.
- **Real-World Example:** A JSON body-parser middleware that converts incoming raw request payloads into accessible JavaScript objects on `req.body`.

Q28: What is the difference between `app.use()` and `app.get()`?

- **Answer:** `app.use()` mounts middleware across all HTTP methods for matching paths, whereas `app.get()` listens exclusively to HTTP GET request methods on a specific route.
- **Real-World Example:** Using `app.use(cors())` globally for all routes, while using `app.get('/profile', ...)` strictly for fetching profile data.

Q29: How do you handle route parameters and query strings in Express?

- **Answer:** Route parameters are captured via `req.params` (e.g., `/users/:id`), while query strings are accessed via `req.query` (e.g., `?sort=asc`).
- **Real-World Example:** Extracting user ID via `req.params.id` to look up a record in a database.

Q30: What is the purpose of the `next()` function in Express middleware?

- **Answer:** It passes control execution forward to the next matching middleware function or route handler in the stack.
- **Real-World Example:** An authentication middleware checking if a token is valid, then calling `next()` to let the user proceed to the dashboard controller.

Q31: What is REST API? How would you design basic CRUD routes for a User resource?

- **Answer:** REST (Representational State Transfer) is an architectural style using standard HTTP verbs for resource management.
- **Real-World Example:**
 - `POST /users` (Create)
 - `GET /users` (Read all)
 - `PUT /users/:id` (Update)
 - `DELETE /users/:id` (Delete)

Q32: What is JSON and why is it the standard data format for APIs in Node.js?

- **Answer:** JSON (JavaScript Object Notation) is a lightweight text-based data interchange format native to JavaScript object structures.
- **Real-World Example:** Sending payload responses directly via `res.json({ status: 'success' })` to frontend React applications.

Q33: What is the difference between PUT and PATCH HTTP methods in a real-world update scenario?

- **Answer:** PUT replaces the entire resource entity with the new payload, whereas PATCH applies partial modifications to update only specified fields.
- **Real-World Example:** Sending a PATCH request to update only a user's email field without wiping out their un-submitted profile address details.

Q34: What are environment variables and why should sensitive data never be hardcoded?

- **Answer:** Environment variables are configuration variables injected securely from the host operating system environment. Hardcoding credentials risks leaking database passwords or API keys if source code is pushed to public repositories like GitHub.
- **Real-World Example:** Storing stripe payment secret keys in hosting platform settings instead of raw code files.

Q35: What is the purpose of the .env file and the dotenv package?

- **Answer:** The .env file stores local development environment variables securely, and the dotenv package loads those variables into process.env during application startup.
- **Real-World Example:** Loading local port configurations and database tokens automatically via `require('dotenv').config()`.

SECTION 2: INTERMEDIATE LEVEL (Q36–Q70)**Q36: Explain the phases of the Node.js Event Loop.**

- **Answer:** The 6 cyclic phases include Timers, Pending Callbacks, Idle/Prepare, Poll (retrieving new I/O events), Check (setImmediate), and Close Callbacks.
- **Real-World Example:** Processing network socket reads inside the Poll phase before routing execution to Check handlers.

Q37: What is the difference between process.nextTick() and setImmediate()?

- **Answer:** `process.nextTick()` queues callbacks to run immediately after the current operation finishes (before the event loop continues), holding higher priority than microtasks, while `setImmediate()` executes during the Check phase of the event loop.
- **Real-World Example:** Prioritizing internal error handling execution before the event loop advances to poll phases.

Q38: What is the difference between setTimeout(fn, 0) and setImmediate(fn)?

- **Answer:** When called from the main module, execution order can be non-deterministic. However, inside an I/O callback, `setImmediate()` always executes before `setTimeout()` because the event loop has already passed the timer phase.
- **Real-World Example:** Scheduling a database post-processing task to run immediately after an incoming I/O query finishes using `setImmediate`.

Q39: What is the microtask queue? How do Promises interact with the Event Loop's phases?

- **Answer:** The microtask queue holds resolved Promise callbacks and `queueMicrotask`. It is drained completely by the V8 engine immediately after the call stack clears and between event loop phases.

- **Real-World Example:** Ensuring `.then()` callbacks execute right after an async function yields data before the next timer ticks.

Q40: Why can a long-running synchronous function block the entire Node.js server?

- **Answer:** Because user JavaScript runs on a single thread, heavy CPU loops occupy the call stack, starving the event loop and preventing it from servicing any other incoming HTTP requests.
- **Real-World Example:** Running a massive cryptographic hashing loop synchronously, causing thousands of concurrent client requests to hang.

Q41: What are Streams in Node.js? Why are they useful for handling large files?

- **Answer:** Streams are data collections that read or write data piece-by-piece (chunks) rather than loading everything into memory at once.
- **Real-World Example:** Uploading a 2GB video file by streaming data chunks directly to cloud storage rather than crashing memory limits via `fs.readFile`.

Q42: What are the four types of streams (Readable, Writable, Duplex, Transform)?

- **Answer:**
 - *Readable:* Stream data can be read from (e.g., `fs.createReadStream`).
 - *Writable:* Stream data can be written to (e.g., `fs.createWriteStream`).
 - *Duplex:* Both readable and writable (e.g., TCP sockets).
 - *Transform:* A duplex stream that modifies data as it's read/written (e.g., zlib compression).
- **Real-World Example:** Compressing a log file on the fly using a Transform stream.

Q43: What is backpressure in streams, and why does it matter for performance?

- **Answer:** Backpressure occurs when a writable stream cannot keep up with data being pushed from a fast readable stream, which can cause memory bloat if buffers overflow.
- **Real-World Example:** Pausing file disk reads temporarily when a slow network client consumer's write buffer fills up.

Q44: What is a Buffer in Node.js? Why is it needed when working with binary data?

- **Answer:** A Buffer is a global class used to represent fixed-length sequences of raw binary data outside V8's standard heap garbage collection memory limits.
- **Real-World Example:** Reading raw bytes from an image file upload to parse file header metadata.

Q45: How would you efficiently stream a large CSV file to a client instead of loading it fully into memory?

- **Answer:** By piping a Readable file stream directly into an Express HTTP response object.
- **Real-World Example:** `fs.createReadStream('huge.csv').pipe(res);`

Q46: What is the difference between operational errors and programmer errors in Node.js?

- **Answer:** Operational errors represent runtime issues (e.g., network failures, missing files), while programmer errors are bugs (e.g., syntax typos, reading properties of undefined).
- **Real-World Example:** Handling a dropped database connection (operational) vs fixing a null reference variable crash (programmer).

Q47: How do you handle uncaught exceptions globally (process.on('uncaughtException'))? Is this a good practice?

- **Answer:** It catches unhandled errors globally, but it is **not** recommended to keep the server running since application state can become corrupted. Best practice is to log the error and perform a graceful shutdown.
- **Real-World Example:** Catching a final system fault before restarting via PM2.

Q48: How do you handle unhandled Promise rejections (process.on('unhandledRejection'))?

- **Answer:** By listening to the unhandledRejection event on the process object to capture promises missing a `.catch()` block.
- **Real-World Example:** Logging unhandled API request rejections before shutting down the container safely.

Q49: How would you design a centralized error-handling middleware in an Express application?

- **Answer:** By creating a middleware function with four parameters (`err`, `req`, `res`, `next`) and placing it at the very bottom of the middleware stack.
- **Real-World Example:** Catching validation or database errors across all controllers in one unified file structure.

Q50: What is the difference between throwing an error and passing it via `next(err)` in Express?

- **Answer:** Throwing an error inside asynchronous route handlers crashes or bypasses Express error routing unless caught, whereas passing `next(err)` explicitly routes the error safely into Express's error middleware stack.
- **Real-World Example:** `if (!user) return next(new Error('Not found'));`

Q51: How would you design authentication in a Node.js app using JWT? What are trade-offs vs session-based auth?

- **Answer:** Generate a signed JSON Web Token upon login containing user claims, store it on the client, and verify it via middleware. JWT is stateless and scalable, but harder to invalidate

instantly compared to server-side session stores.

- **Real-World Example:** Authenticating a React single-page app via Bearer tokens in headers.

Q52: How would you implement rate limiting on an API to prevent abuse?

- **Answer:** Using packages like `express-rate-limit` backed by Redis to track IP request counts over sliding time windows.
- **Real-World Example:** Limiting login attempts to 5 requests per 15 minutes to prevent brute-force attacks.

Q53: How do you handle file uploads in Express? What real-world validations are needed?

- **Answer:** Using the `multer` middleware. Validations should check file size limits, MIME types, and file extensions.
- **Real-World Example:** Allowing only profile image uploads (`.jpg/.png`) under 2MB.

Q54: How would you structure a scalable Node.js project? Why does this separation matter?

- **Answer:** Using a layered architecture (Routes -> Controllers -> Services -> Models/Data Access). This separates concerns, making code testable and maintainable.
- **Real-World Example:** Placing business logic inside a service layer rather than crowding route handlers.

Q55: What is middleware chaining, and how would you use it for request validation, authentication, and logging?

- **Answer:** Executing multiple middleware functions sequentially in an array or comma-separated list on a route.
- **Real-World Example:** `router.post('/checkout', logger, verifyAuth, validateCart, checkoutController);`

Q56: How do you connect Node.js to a database? What is connection pooling?

- **Answer:** Using database drivers/ORMs (like Mongoose or Prisma). Connection pooling maintains a reusable cache of open database connections, avoiding the overhead of opening a new TCP connection for every query.
- **Real-World Example:** Reusing a pool of 10 PostgreSQL connections across 500 concurrent web requests.

Q57: How would you prevent SQL Injection / NoSQL Injection in a Node.js application?

- **Answer:** Using parameterized queries, ORM query builders (Prisma, Sequelize), or sanitizing inputs to prevent injection attacks.
- **Real-World Example:** Passing parameters as arrays (`[username]`) instead of raw string interpolation in SQL queries.

Q58: How would you implement pagination for an API returning thousands of records?

- **Answer:** Using cursor-based or limit-offset pagination (LIMIT and OFFSET clauses in SQL queries).
- **Real-World Example:** Fetching items in chunks of 20 (/api/items?limit=20&page=2).

Q59: How would you design a caching layer (e.g., Redis) to reduce database load?

- **Answer:** Check the Redis cache before querying the database; if data exists (cache hit), return it; otherwise, fetch from the database, store it in Redis with an expiration TTL, and return it.
- **Real-World Example:** Caching user profile feeds for 5 minutes to reduce database read pressure.

Q60: How do you implement CORS in an Express app, and why is it needed?

- **Answer:** Using the cors middleware package (app.use(cors())). It is needed because modern browsers enforce the Same-Origin Policy, blocking cross-origin frontend-to-backend API calls by default.
- **Real-World Example:** Allowing a React app running on localhost:3000 to access an Express API running on localhost:5000.

Q61: What is unit testing vs integration testing in a Node.js context?

- **Answer:** Unit testing tests individual functions/modules in isolation (mocking dependencies), while integration testing tests how multiple modules or systems (database, API endpoints) interact together.
- **Real-World Example:** Testing a password hashing utility function (unit) vs testing a full user registration HTTP endpoint with a test database (integration).

Q62: What tools are commonly used for testing Node.js applications?

- **Answer:** Jest or Mocha for test runners, Chai for assertions, and Supertest for HTTP endpoint assertions.
- **Real-World Example:** Writing a Supertest assertion to verify POST /login returns a 200 status code and a JWT token.

Q63: How would you mock a database call in a unit test?

- **Answer:** Using testing framework features like jest.mock() to intercept and stub database driver model methods.
- **Real-World Example:** Mocking User.find() to return a fake mock user object without hitting a live database.

Q64: What is the purpose of a linter (ESLint) in a real-world Node.js team project?

- **Answer:** To statically analyze code to catch syntax errors, enforce consistent code styling, and prevent bad coding patterns across team members.

- **Real-World Example:** Automatically failing a CI build if a developer leaves an unused variable or missing semicolon in code.

Q65: How do you handle environment-specific configurations (dev, staging, production)?

- **Answer:** By using configuration modules or environment variables loaded based on `NODE_ENV` settings.
- **Real-World Example:** Connecting to a local SQLite database in development, but a clustered AWS RDS PostgreSQL instance in production.

Q66: What are common security vulnerabilities in Node.js apps?

- **Answer:** Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL/NoSQL injection, and insecure deserialization.
- **Real-World Example:** Sanitizing user input inputs to prevent script injection in chat message boards.

Q67: Why should you use helmet middleware in an Express app?

- **Answer:** Helmet helps secure Express apps by setting various HTTP headers (like Content-Security-Policy, X-Frame-Options) to protect against well-known web vulnerabilities.
- **Real-World Example:** Preventing clickjacking attacks by setting protective framing headers automatically.

Q68: How do you securely store and compare passwords?

- **Answer:** Using cryptographic hashing functions with salts, such as the `bcrypt` library. Never store plain text passwords.
- **Real-World Example:** Hashing a user's password during signup via `bcrypt.hash(password, 10)` before saving to the database.

Q69: What is the principle of least privilege, and how does it apply to database user permissions?

- **Answer:** Giving users or application services only the absolute minimum permissions required to perform their tasks.
- **Real-World Example:** Connecting the Node.js application to PostgreSQL using a database user role that has `SELECT/INSERT` permissions, but dropping administrative `DROP DATABASE` access capabilities.

Q70: How would you protect an API from a DDoS-style abuse pattern at the application level?

- **Answer:** Implementing rate limiting, IP request throttling, payload size limits, and leveraging cloud-level Web Application Firewalls (WAF) like Cloudflare.
- **Real-World Example:** Dropping connections from an IP address flooding the login route with 1000 requests per second.

SECTION 3: ADVANCED LEVEL (Q71–Q100)

Q71: Node.js is single-threaded, so how does it handle multiple concurrent requests efficiently? Explain the role of libuv.

- **Answer:** Libuv is the underlying C++ multi-platform library that implements the event loop and manages background thread pools and asynchronous OS-level I/O polling.
- **Real-World Example:** Offloading file system reads to libuv's thread pool while the main thread keeps accepting web traffic.

Q72: What is the Worker Threads module, and when would you use it over the main Event Loop?

- **Answer:** The `worker_threads` module allows running JavaScript in parallel on multiple CPU threads. It is used for heavy CPU-bound tasks to prevent blocking the main event loop.
- **Real-World Example:** Offloading high-resolution image resizing or video transcoding to a worker thread.

Q73: What is the Cluster module, and how does it help utilize multi-core CPUs?

- **Answer:** The `cluster` module allows spawning multiple Node.js worker processes (instances) that share the same server port, distributing incoming traffic across multiple CPU cores.
- **Real-World Example:** Spawning 4 worker processes on a 4-core server to scale web throughput linearly.

Q74: What is the difference between horizontal scaling and vertical scaling for a Node.js application?

- **Answer:** Vertical scaling means adding more power (CPU/RAM) to a single server instance, whereas horizontal scaling means adding more server instances behind a load balancer.
- **Real-World Example:** Deploying 10 cluster instances across an AWS auto-scaling group (horizontal) vs upgrading an EC2 server tier (vertical).

Q75: How would you profile and identify a memory leak in a Node.js production application?

- **Answer:** By taking heap snapshots using tools like Chrome DevTools or Clinic.js, then comparing snapshots over time to find objects that fail to be garbage collected.
- **Real-World Example:** Finding a global array growing infinitely with unclosed event listeners.

Q76: What is a common cause of memory leaks in Node.js?

- **Answer:** Unbounded caches, global variable accumulation, and unremoved event listeners/timers.
- **Real-World Example:** Storing user session states inside a local JavaScript object map without an eviction policy or expiration limit.

Q77: How does the V8 garbage collector work, and how can poor coding patterns increase GC pressure?

- **Answer:** V8 uses a generational garbage collector (Scavenge for young generation, Mark-Sweep-Compact for old generation). Frequent creation of short-lived objects increases GC frequency ("GC pressure"), causing micro-pauses in execution.
- **Real-World Example:** Allocating heavy object graphs inside tight performance-critical loops.

Q78: How would you handle a CPU-intensive task without blocking the Event Loop?

- **Answer:** By offloading the task to Worker Threads, child processes, or external microservice worker queues.
- **Real-World Example:** Generating large PDF reports inside a separate worker thread pool.

Q79: How would you design a real-time chat application using Node.js?

- **Answer:** Using WebSockets (via libraries like `Socket.io`) for persistent, bi-directional communication channels instead of HTTP polling.
- **Real-World Example:** Broadcasting chat messages instantly to all connected clients inside a chat room channel.

Q80: What is the difference between REST and GraphQL? When would you choose GraphQL?

- **Answer:** REST exposes multiple distinct resource endpoints with fixed data structures, while GraphQL provides a single endpoint where clients query exact data requirements. Choose GraphQL when dealing with complex, deeply nested relational data schemas to prevent over-fetching.
- **Real-World Example:** Fetching a user profile along with their last 3 posts and comments in a single GraphQL query instead of 3 separate REST roundtrips.

Q81: How would you design a microservices architecture in Node.js? How do services communicate?

- **Answer:** Breaking an application into independent domain-specific services communicating via REST APIs, gRPC, or asynchronous message queues (RabbitMQ/Kafka).
- **Real-World Example:** Separating an Order service from a Payment service using message brokers.

Q82: What role do message queues play in a Node.js microservices system? Give a real-world use case.

- **Answer:** They decouple services, provide asynchronous buffering, and ensure fault-tolerant message delivery.
- **Real-World Example:** Pushing an order-placed event to RabbitMQ so an email notification service can process it asynchronously.

Q83: How would you implement an event-driven architecture using Node's EventEmitter?

- **Answer:** By extending the `EventEmitter` class to emit custom application events and attach listener callbacks.
- **Real-World Example:** Emitting an `'userRegistered'` event to trigger welcome email and analytics tracking listeners.

Q84: How would you design a job queue system for background tasks using something like BullMQ?

- **Answer:** By leveraging Redis-backed queue libraries like BullMQ to enqueue, schedule, retry, and process heavy background tasks reliably.
- **Real-World Example:** Processing nightly report generation jobs at 2 AM.

Q85: How would you handle distributed transactions across multiple microservices?

- **Answer:** By using patterns like the Saga Pattern (orchestration or choreography) with compensating transactions to ensure eventual consistency.
- **Real-World Example:** Rolling back a payment charge if the inventory reservation service fails downstream.

Q86: How would you design an API Gateway in front of multiple Node.js microservices?

- **Answer:** By implementing a reverse proxy (using tools like Nginx, Kong, or a custom Express proxy) to handle cross-cutting concerns like authentication, rate limiting, and request routing.
- **Real-World Example:** Routing `/api/users` traffic to the User microservice and `/api/orders` to the Order microservice.

Q87: What is idempotency, and why is it critical when designing payment or order APIs?

- **Answer:** Idempotency ensures that making the same API request multiple times produces the same result without duplicate side-effects.
- **Real-World Example:** Using idempotency keys so that if a user clicks "Pay" twice due to a lagging network, their credit card is only charged once.

Q88: How would you version an API in a real-world Node.js application?

- **Answer:** By including version prefixes in URL routes (`/api/v1/users`) or via custom HTTP request headers.
- **Real-World Example:** Maintaining `/api/v1/` while introducing breaking changes in `/api/v2/`.

Q89: What is the difference between `require()` and `dynamic import()`?

- **Answer:** `require()` is synchronous and loads CommonJS modules statically, whereas `dynamic import()` is asynchronous, returns a Promise, and supports loading ES modules conditionally at runtime.

- **Real-World Example:** Loading a heavy localization language pack dynamically only when a user changes their language setting.

Q90: Explain how Node.js resolves module paths (node_modules lookup algorithm).

- **Answer:** Node.js checks core modules first, then looks in the current directory's `node_modules`, and if not found, traverses up parent directories recursively until it hits the root filesystem.
- **Real-World Example:** Requiring `lodash` inside a nested subfolder and having Node successfully locate it in the project root's `node_modules`.

Q91: What is the Event Emitter pattern, and how is it used internally in core modules?

- **Answer:** It's an observer design pattern where objects emit named events that trigger subscribed listeners. Core modules like `http` (servers emitting `'request'`) and `stream` (streams emitting `'data'`, `'end'`) use it extensively.
- **Real-World Example:** Listening to incoming data chunks on a readable stream via `stream.on('data', chunk => { ... })`.

Q92: What is the difference between deep copy and shallow copy in JavaScript objects?

- **Answer:** A shallow copy copies top-level properties by reference for nested objects, whereas a deep copy recursively clones all nested objects entirely. Mutating shared nested configs in shallow copies causes subtle bugs.
- **Real-World Example:** Using `structuredClone()` to perform a true deep copy of a complex configuration object.

Q93: How does Node.js handle child processes (spawn, exec, fork)?

- **Answer:**
 - `spawn`: Streams command output data (good for large data sets).
 - `exec`: Buffers output into memory and passes it to a callback.
 - `fork`: A specialized variant of `spawn` dedicated to spawning new Node.js child processes for IPC communication.
- **Real-World Example:** Spawning a shell command utility script via `child_process.spawn`.

Q94: What is the difference between spawn and exec in terms of buffering output?

- **Answer:** `exec` buffers the entire command output in memory up to a max buffer limit (crashing if exceeded), while `spawn` streams data incrementally as it arrives.
- **Real-World Example:** Using `spawn` instead of `exec` when reading a massive log-dump command output to prevent memory buffer overflow errors.

Q95: What is graceful shutdown, and how do you implement it?

- **Answer:** Graceful shutdown stops accepting new incoming connections, finishes processing in-flight requests, closes database connection pools, and exits cleanly.
- **Real-World Example:** Catching `SIGTERM` signals from Kubernetes during container rolling deployments to drain traffic gracefully before shutting down.

Q96: How would you implement health checks for a Node.js service running in Kubernetes?

- **Answer:** By exposing `/healthz` (liveness) and `/readyz` (readiness) HTTP endpoints that verify database connectivity and internal states.
- **Real-World Example:** Returning HTTP 500 on the readiness probe if Redis goes down, instructing Kubernetes to stop routing traffic to that pod.

Q97: What are the trade-offs of using Docker to containerize a Node.js application?

- **Answer:** Trade-offs include consistent environment parity across deployment stages (pro), versus added container image build complexities, multi-stage build optimizations, and slight container virtualization overhead (con).
- **Real-World Example:** Using multi-stage Docker builds to strip out `devDependencies` from production runner images.

Q98: How would you implement structured logging and distributed tracing across microservices?

- **Answer:** By using JSON loggers (like Winston or Pino) and passing a unique `X-Correlation-ID` header across all downstream microservice HTTP requests.
- **Real-World Example:** Tracing a single user checkout request through 4 different microservices using a shared correlation ID in log outputs.

Q99: What is Server-Sent Events (SSE), and how does it compare to WebSockets?

- **Answer:** SSE enables a server to push real-time data updates unidirectionally over a single HTTP connection to the client, whereas WebSockets provide full bi-directional, two-way communication.
- **Real-World Example:** Using SSE for live sports score tickers or stock price notification feeds where client-to-server messaging is unnecessary.

Q100: If you were designing a real-world food delivery backend (like Zomato/Uber Eats) in Node.js, how would you architect it?

- **Answer:**
 - **Architecture:** Microservices split by domain (Restaurants, Orders, Payments, Delivery Tracking, Notifications).
 - **Real-Time Updates:** WebSockets for live rider location tracking and order status updates.
 - **Payments & Reliability:** Idempotent payment APIs, BullMQ for background notification dispatching, and Kafka message brokers for event-driven order processing pipelines.

- **Real-World Example:** When an order is placed, an event fires to queue restaurant confirmation, trigger payment processing, and open a WebSocket channel for live delivery tracking.
-